Da una chiamata API a un team di agenti

Anthropic SDK e Claude Agent SDK: costruire un sistema multiagente



Nicola Procopio - Data & AI Scientist @

Claude Community Cosenza · 28 Ottobre 2026

Il percorso di oggi



2'

1. Le due SDK

Mattoni e motore



1'

2. Architettura

Orchestrator e specialisti



4'

3. La cartella .claude

Struttura e gerarchia



4'

4. Lo spawn

Delega e comunicazione



3'

5. Tool e MCP

Quali e quanti



3'

6. Errori ed escalation

Retry e umano nel ciclo



1'

7. Tips & tricks

Quello che ho imparato

20'

in tutto, con un minuto di apertura e uno di chiusura

Due SDK, due livelli

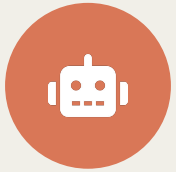


Anthropic SDK

I mattoni

- Messages API e streaming
- Tool use: tu definisci gli schemi e esegui i tool
- Prompt caching, batch, output strutturati

Il ciclo agentico lo scrivi tu



Claude Agent SDK

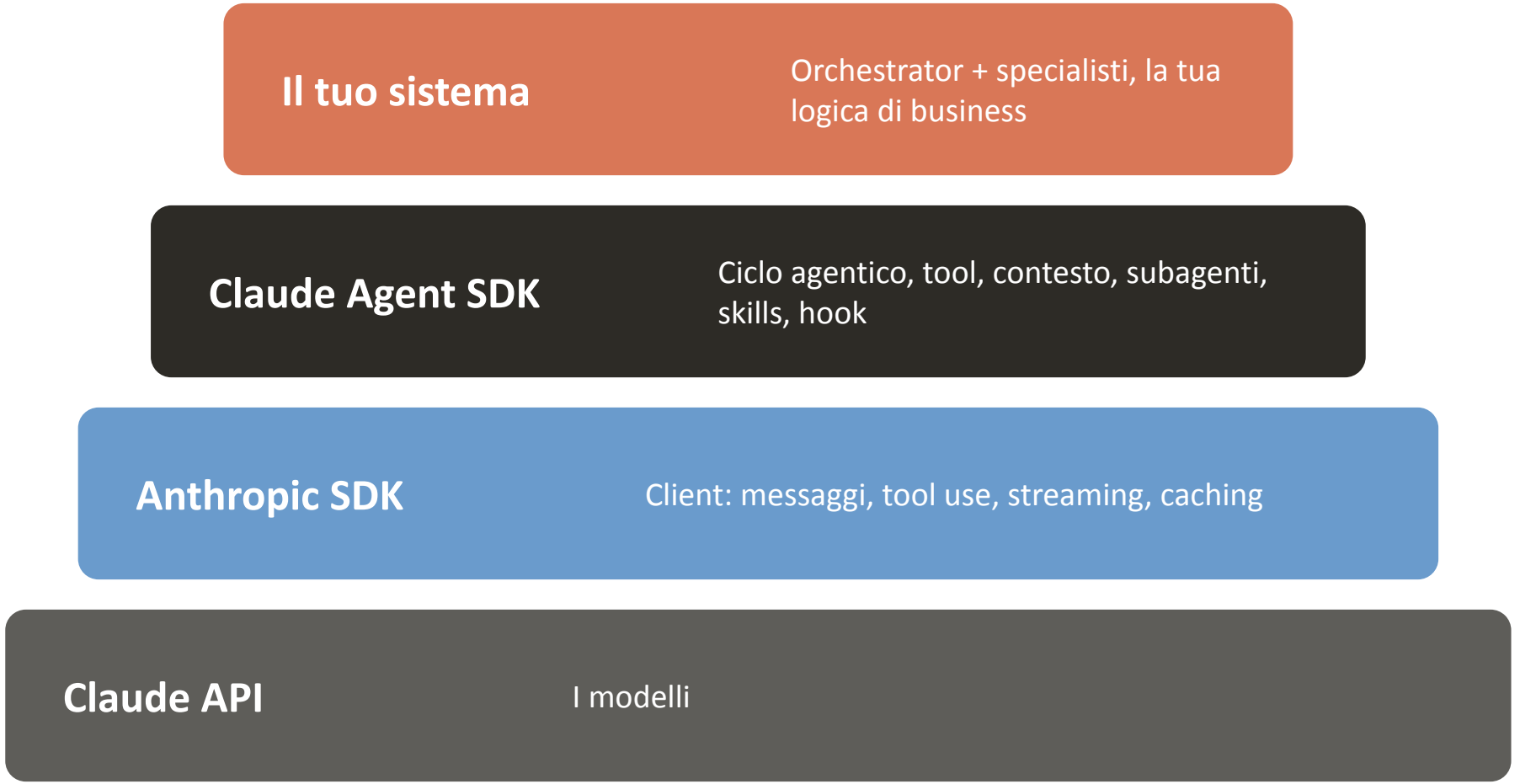
Il motore di Claude Code, come libreria

- Ciclo agentico e gestione del contesto inclusi
- Tool pronti: file, bash, ricerca, web
- Subagenti, skills, hook, permessi, MCP

Il ciclo è incluso: tu configuri

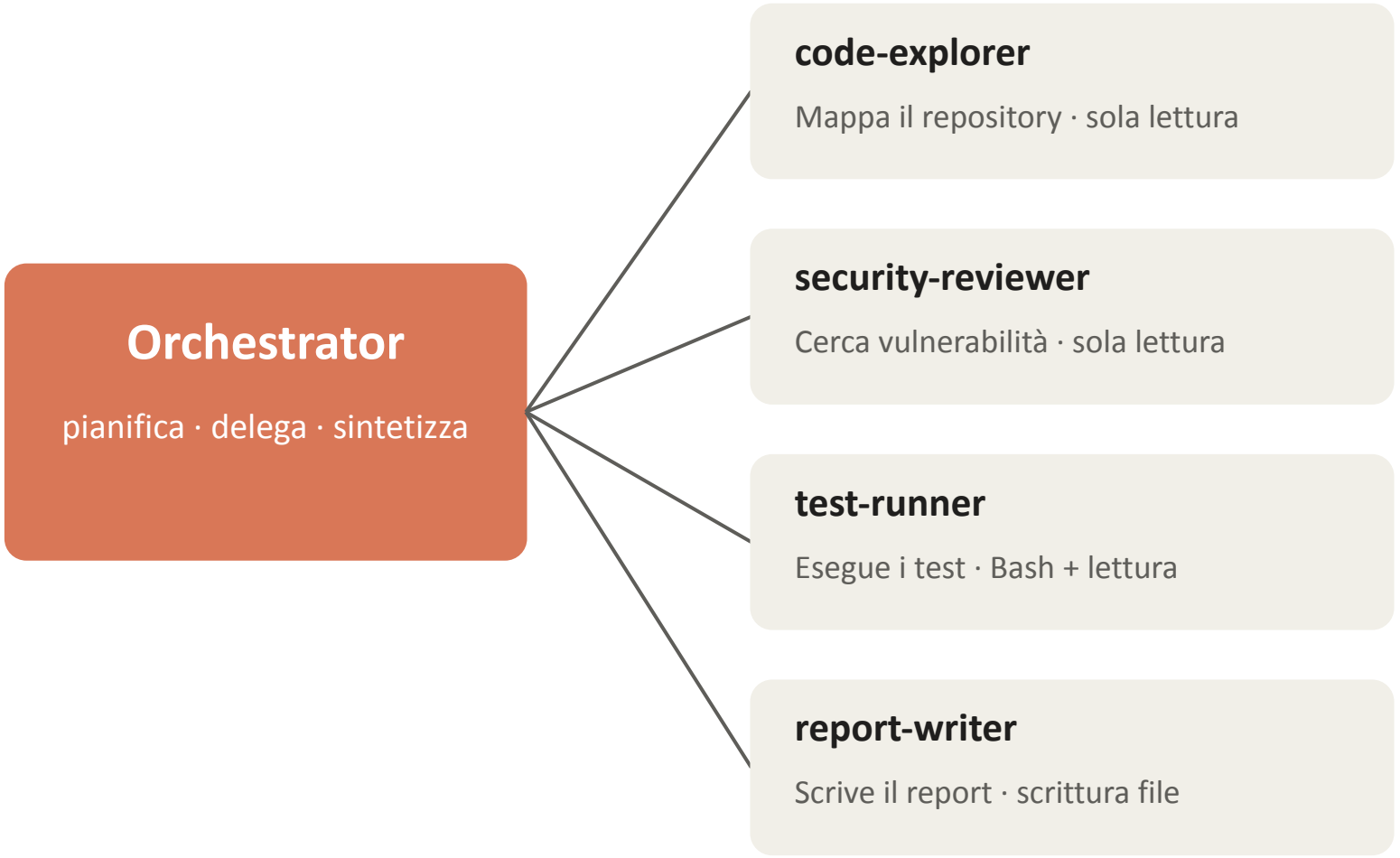
Controllo totale su un flusso semplice → **Anthropic SDK** · Autonomia su compiti aperti → **Agent SDK**

Lo stesso stack, un livello sopra l'altro



Più sali, più comportamento ti arriva già pronto

Orchestrator e specialisti



Contesti piccoli

Ogni agente vede solo ciò che serve al suo compito

Un ruolo, pochi tool

Lo specialista è bravo perché è stretto

Solo risultati

L'orchestrator riceve sintesi, non il percorso

La cartella .claude

```
progetto/
├── CLAUDE.md
├── CLAUDE.local.md          (fuori da git)
├── .mcp.json
└── .claude/
    ├── settings.json
    ├── settings.local.json (fuori da git)
    ├── agents/
    │   └── security-reviewer.md
    ├── skills/
    │   └── report-pdf/
    │       ├── SKILL.md
    │       └── scripts/
    ├── rules/
    │   └── python-style.md
    └── commands/
```

Quando entra nel contesto?

SEMPRE

CLAUDE.md e rules

Convenzioni, comandi, architettura. Tienili corti: costano a ogni turno.

SU RICHIESTA

skills

All'avvio solo nome e descrizione; istruzioni e script quando servono.

CONTESTO SEPARATO

agents

Ogni subagente ha la sua finestra di contesto.

CONFIGURAZIONE

settings.json e .mcp.json

Permessi, hook, modello, server MCP condivisi.

Chi vince? Progetto contro locale

settings.json: vince il livello più alto

1	**Managed**	Policy dell'organizzazione	*non sovrascrivibile*
2	**Riga di comando**	--settings, solo questa sessione	*temporaneo*
3	**Locale**	.claude/settings.local.json	*fuori da git*
4	**Progetto**	.claude/settings.json	*in git, per il team*
5	**Utente**	~/.claude/settings.json	*tutti i tuoi progetti*

Stessa chiave: vince il livello più alto. Le liste, come i permessi, si sommano.



CLAUDE.md: si sommano

Utente, progetto e CLAUDE.local.md: l'agente li legge tutti. Se si contraddicono non c'è una regola fissa: scrivi la precedenza nel file più specifico.



Agenti con lo stesso nome

Progetto batte utente. Nell'SDK l'agente definito nel codice batte i file.



Skill con lo stesso nome

Personale batte progetto: l'ordine è opposto a quello degli agenti.

Lo sviluppatore con preferenze diverse usa settings.local.json e CLAUDE.local.md: restano fuori da git e valgono solo per lui.

Dalla cartella all'SDK



Agenti: da file o da codice

Markdown in .claude/agents/ oppure definiti nel codice, l'opzione consigliata per le app SDK. A parità di nome vince il codice.



Skills: solo da disco

Una cartella con SKILL.md più script e risorse. Non esiste un'API per registrarle: vanno su filesystem.



CLAUDE.md e rules

Gli stessi file valgono per Claude Code interattivo e per i tuoi agenti SDK: una sola fonte di verità per il team.



Attenzione: di default l'Agent SDK carica la cartella come la CLI: utente, progetto e locale. In produzione decidi tu con le setting sources, o nessuna per isolare l'agente.

Come avviene lo spawn di un subagente

1

Legge le description

Vede ruolo e "quando usarmi" di ogni specialista

2

Delega

Chiama il tool Agent con un prompt di incarico

3

Contesto pulito

Il subagente parte da zero: suo prompt, suoi tool, suo modello

4

Lavora da solo

Letture e tool call restano dentro il subagente

5

Restituisce

Al padre torna solo il messaggio finale



La description è il router. Scrivila come un annuncio di lavoro: cosa fa lo specialista e quando chiamarlo. Se l'orchestrator non delega, il problema è quasi sempre lì.

Cosa porta con sé un subagente

Riceve

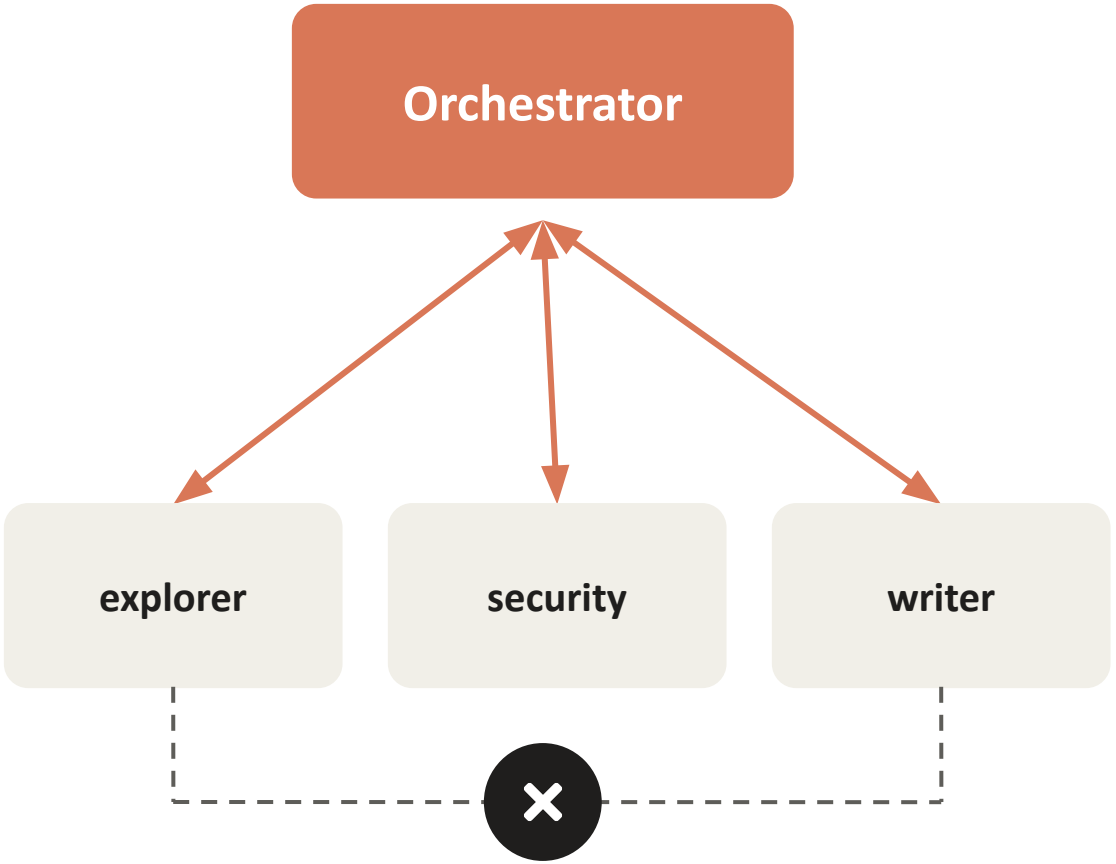
- Il suo system prompt
- Il prompt di incarico dell'orchestrator
- Il CLAUDE.md di progetto
- I tool che gli hai assegnato

Non riceve

- La cronologia della conversazione
- I risultati dei tool del padre
- Il system prompt del padre
- Le skill, se non le precarichi

Il prompt di incarico è l'unico ponte. Percorsi dei file, messaggi di errore, decisioni già prese: se servono al subagente, vanno scritti lì.

Si comunica solo passando dall'alto



niente scambi diretti tra specialisti

Niente domande tra pari

Se a uno specialista serve qualcosa da un altro, lo scrive nel risultato. Decide l'orchestrator.

Niente memoria condivisa

Tutto ciò che serve arriva nel prompt di incarico, tutto ciò che produce torna nel risultato.

Lo stato vive in alto

L'orchestrator raccoglie, decide e ri-delega: un solo punto di verità, un solo posto da debuggare.

In pratica: agli specialisti niente tool Agent né SendMessage, profondità di spawn 1, nessuna memoria condivisa.

Parallelismo e limiti

I subagenti partono in background: i compiti indipendenti finiscono nel tempo del più lento, non nella somma. E possono generare altri subagenti, quindi una richiesta può diventare un albero.

3

livelli di annidamento

di default. Imposta 1 per un'architettura a un solo livello

20

subagenti in parallelo

di default. Oltre, lo spawn viene rifiutato finché qualcuno non finisce

$

tetto di spesa

per query: conta anche i subagenti e ferma tutto al limite

Tre famiglie di tool

Built-in

Read, Edit, Write, Bash, Grep, Glob, ricerca web. Pronti all'uso, già integrati con i permessi.

Custom in-process

Le tue funzioni Python esposte come server MCP nello stesso processo. Nessun server da gestire.

Server MCP esterni

GitHub, database, Slack, Drive: qualsiasi server MCP, locale o remoto, dichiarato in .mcp.json o nel codice.

Naming: i tool MCP si chiamano mcp__server__tool, quindi puoi abilitarli o escluderli per singolo tool o per server intero.

Quanti tool per agente?

Pochi

Orchestrator

Il tool Agent e la lettura. Se fa tutto da solo, non sta orchestrando.

3–7 mirati

Specialista

Least privilege: solo quelli del suo mestiere.

Dividi

Più di 7

Segnale che l'agente fa due mestieri: separalo in due specialisti.

Indicazioni pratiche, non soglie ufficiali

Combinazioni tipiche

Ruolo	Tool
Analisi in sola lettura	Read, Grep, Glob
Esecuzione test	Bash, Read, Grep
Modifica del codice	Read, Edit, Write, Grep, Glob

Ogni tool ha un costo:
lo schema occupa contesto e ogni tool in più rende più difficile scegliere quello giusto. Un tool escluso, per quell'agente, semplicemente non esiste.

Errori: cosa si riprova e cosa no

Transitori

Riprova con backoff

- Rate limit (429)
- Sovraccarico (529)
- Errori server (5xx)
- Timeout, connessione caduta

Li gestisce già l'Anthropic SDK: retry automatici con backoff esponenziale, configurabili.

Recuperabili

Riprova cambiando qualcosa

- Output che non rispetta il contratto
- Tool fallito per input sbagliato
- Risultato parziale (turni finiti)
- Contesto troppo lungo

Ripetere identico non serve: l'orchestrator ri-delega con un incarico corretto o più stretto.

Permanenti

Non riprovare

- Richiesta non valida (400)
- Autenticazione e permessi (401, 403)
- Risorsa inesistente (404)
- Azione negata da utente o hook, budget finito

Riprovare dà lo stesso risultato e costa. Ci si ferma e si risale.

Di business

Non è un guasto: decide il dominio

- Regola violata: importo oltre soglia, cliente bloccato
- Stato incoerente: ordine già evaso
- Dati in conflitto tra fonti
- Esito valido ma negativo

Il sistema ha funzionato. È un esito: decidono le regole o l'umano.

Regola d'oro: riprova solo azioni idempotenti e metti un tetto ai tentativi. Rileggere un file va bene, rimandare un'email no.

L'errore risale, non si nasconde

Il rapporto di errore dello specialista

esito	ok · parziale · fallito
tipo	transitorio · recuperabile · permanente · di business
cosa ho provato	passi fatti e tentativi già spesi
cosa ho ottenuto	il lavoro parziale, mai buttato
cosa serve	l'informazione o il permesso mancante

Lo specialista non decide il retry: descrive e restituisce.

Cosa fa l'SDK da solo

- Subagente interrotto da un errore API: torna l'output parziale con una nota
- In background: marcato come fallito, con errore e ultimo output

Cosa decide l'orchestrator

Ri-delega | **Cambia strada**

Usa il parziale | **Scala all'umano**

Escalation all'umano

Specialista › Orchestrator › Umano

Lo specialista segnala, l'orchestrator scala. Le domande all'utente non sono disponibili nei subagenti: anche qui si passa dall'alto.

Quando

- Azioni irreversibili: cancellare, pubblicare, inviare, pagare
- Errore permanente o di business, tentativi esauriti
- Ambiguità che cambia il risultato
- Specialisti in disaccordo tra loro
- Budget o tempo quasi finiti

Come

Approvazione dei tool

Un callback riceve la richiesta: approvi, approvi con modifiche o neghi spiegando perché.

Domande chiarificatrici

AskUserQuestion: l'agente propone le opzioni, l'umano sceglie.

Hook deterministici

Regole in codice prima di ogni tool: consenti, nega o rimanda a dopo.

Notifiche e attese lunghe

Avvisi su Slack o email; la sessione si salva e riprende alla risposta.

Tips & tricks (1/2)

Modelli misti

Modello potente per l'orchestrator, uno veloce ed economico per gli specialisti ripetitivi. Il modello si sceglie per agente.

Contratto di output

Chiedi a ogni subagente lo stesso formato: esito, evidenze, file toccati. La sintesi diventa molto più affidabile.

Chiamalo per nome

"Usa l'agente security-reviewer" forza la delega quando la description da sola non basta.

Hook come guardrail

Blocca i comandi pericolosi, registra ogni tool call, valida l'output prima che torni al padre.

Tips & tricks (2/2)



Metti un tetto

Budget, profondità, concorrenza, numero massimo di turni. I modelli più recenti delegano volentieri: i limiti li decidi tu.



Riprendi un subagente

Con resume continua da dove si era fermato, con tutta la sua storia: utile per le domande di follow-up.



Fatti scrivere gli agenti

Chiedi a Claude Code di generare la definizione di un subagente, poi rifinisci description e tool a mano.



Prompt caching

System prompt lunghi e stabili vanno in cache: costi e latenza scendono, soprattutto con molti spawn.

Quattro cose da portarsi a casa

1. Il contesto è la risorsa più preziosa
2. Le description sono il router
3. Meno tool, ma quelli giusti
4. Si comunica solo passando dall'alto

Domande?

docs.claude.com · code.claude.com/docs/en/agent-sdk

Grazie per l'attenzione

